O **padrão Strategy** é um padrão de projeto comportamental que permite definir uma família de algoritmos, encapsular cada um em sua própria classe e torná-los intercambiáveis. Com ele, você evita estruturas gigantescas de `if-else` ou `switch-case` ao delegar o comportamento a uma interface comum. [1, 2]

---

Como ele é estruturado em Java

A implementação clássica envolve três partes principais:

1. **A Interface (Estratégia):** Define o contrato (o método) que todos os algoritmos devem seguir.
2. **As Implementações Concretas:** Classes que implementam a interface, cada uma com uma variação específica do algoritmo.
3. **O Contexto:** A classe que armazena uma referência para um objeto da interface e delega a execução para ele (pode permitir a troca da estratégia em tempo de execução). [1, 2, 3, 4]

---

Exemplo prático: Sistema de Pagamento

1. Criando a Interface Strategy

java

```
public interface PaymentStrategy {
    void pay(int amount);
}
```

Use o código com cuidado.

2. Criando as Estratégias Concretas

java

```
public class CreditCardStrategy implements PaymentStrategy {
    private String cardNumber;

    public CreditCardStrategy(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    @Override
    public void pay(int amount) {
        System.out.println("Pago R$ " + amount + " com Cartão de Crédito (" +
cardNumber + ").");
    }
```

```
    }

    public class PixStrategy implements PaymentStrategy {
        private String chavePix;
```

Use o código com cuidado.

3. Criando o Contexto

java

```
public class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    // Permite definir ou trocar a estratégia
    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(int amount) {
        if (paymentStrategy == null) {
            throw new IllegalStateException("Nenhuma estratégia de pagamento
definida!");
        }
        paymentStrategy.pay(amount);
    }
}
```

Use o código com cuidado.

4. Usando na prática

java

```
public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        // Pagando com Cartão
        cart.setPaymentStrategy(new CreditCardStrategy("1234-5678-9012-
3456"));
        cart.checkout(150);

        // Mudando a estratégia dinamicamente para PIX
        cart.setPaymentStrategy(new PixStrategy("suachave@pix.com"));
        cart.checkout(80);
```

```
        }
    }
```

Use o código com cuidado.

---

Alternativa moderna no Java 8+

A partir do Java 8, se a sua estratégia for uma **Interface Funcional** (um único método abstrato), você pode simplificar e passar a lógica diretamente usando **Expressões Lambda**, dispensando a criação de várias classes pequenas: [1]

java

```
public class ShoppingCartFunctional {
    private Consumer<Integer> paymentStrategy;

    public void setPaymentStrategy(Consumer<Integer> paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void checkout(int amount) {
        paymentStrategy.accept(amount);
    }
}

// Uso com Lambda:
ShoppingCartFunctional cart = new ShoppingCartFunctional();
cart.setPaymentStrategy(valor -> System.out.println("Pago R$ " + valor + " via
Pix (Lambda)"));
cart.checkout(100);
```

Use o código com cuidado.

Você pode aprofundar os conceitos visuais e estruturais no guia completo do Refactoring.Guru Strategy em Java.